Journal of Information Security and Applications 93 (2025) 104188

Contents lists available at ScienceDirect

Journal of Information Security and Applications

journal homepage: www.elsevier.com/locate/jisa

Evaluating security pitfalls of ultra-lightweight IoT authentication: A critical analysis of permutation functions

Iman Mirzaali [a], Alireza Javadi [b], Sadegh Sadeghi [c], Payman Pahlevani [b], Nasour Bagheri [a,d,*], Ygal Bendavid [e], Samad Rostampour [f,e]

[a] Department of Communication Engineering, Faculty of Electrical Engineering, Shahid Rajaee Teacher Training University, Tehran, Islamic Republic of Iran
[b] Department of Computer Science and Information Technology, Institute for Advanced Studies in Basic Sciences (IASBS), Zanjan, 45137-6673, Iran
[c] Department of Mathematics, Institute for Advanced Studies in Basic Sciences (IASBS), Zanjan, 45137-66731, Iran
[d] School of Computer Science (SCS), Institute for Research in Fundamental Sciences (IPM), Tehran, Iran
[e] Department of Analytics Operations and Technology, School of Management, Université du Québec à Montréal (UQAM), Montreal QC. H2X 3X2, Canada
[f] Department of Computer Science, Vanier College, Montreal, QC, Canada

ARTICLE INFO

Keywords:
Authentication
IoT
RFID
Ownership transfer
Security analysis

ABSTRACT

The resource constraints in IoT devices necessitate using protocols with low complexity and high security. One popular method for reducing the weight of protocols is using permutation functions. This paper presents a comprehensive security evaluation of a recently proposed ultra-lightweight authentication protocol (*JIOT.2023.3267501*). The protocol aims to provide a mechanism for secure ownership transfer of RFID objects. Our analysis assesses the protocol's vulnerability to a full secret disclosure attack by exploiting weaknesses in the underlying permutation. Specifically, we examine using $(l+1)$-sessions with a secret length denoted as l. Additionally, we identify the protocol's susceptibility to desynchronization attacks and the potential compromise of the new owner's privacy by the previous owner. Furthermore, we demonstrate that the internal structure of permutation functions is inherently vulnerable and can create weak points even within protocols that otherwise have a robust structure. The attack methodology is applied in both computational and hardware forms within a real-world setting, employing three Raspberry Pis and a wireless network. The details of the attack are also demonstrated in a video that comprehensively outlines all the steps.

1. Introduction

The Internet of Things (IoT) has become an essential subject in the technology landscape, spanning various sectors like industry, healthcare, and retail. System designers are actively working on creating cost-effective and lightweight devices to drive user adoption and encourage the widespread use of IoT. Their objective is to offer budget-friendly solutions accessible to a broad audience. As a result, IoT device design has shifted towards developing lightweight and ultra-lightweight systems, with "weight" encompassing energy consumption, bandwidth, and hardware complexity.

However, adopting devices with minimal complexity and eliminating sophisticated hardware circuits to cut costs and energy consumption introduces security risks, making these systems susceptible to potential threats from hackers. Consequently, designing a lightweight system with robust security has become a primary objective in IoT-based system design. Considering various technologies such as RFID (Radio Frequency Identification), Bluetooth, and Wi-Fi in IoT systems, a security solution offering greater flexibility and applicability to diverse infrastructures will likely gain popularity.

Given RFID's significant role in the IoT landscape and the extensive research conducted in recent years on the security of these systems, researchers have proposed various security solutions applicable to areas like authentication protocols and ownership transfer. They aim to design lightweight methods that deliver optimal performance in both the physical and edge layers. Ultra-lightweight approaches have gained popularity among these methods because they leverage minimal hardware and rely on simple logical functions such as shift, XOR, and permutation operations to ensure security.

It can be argued that designing ultra-lightweight protocols is more challenging than designing lightweight protocols. The challenge stems from lightweight protocols predominantly using standard encryption methods like ECC (Elliptic Curve Cryptography) and AES (Advanced

* Corresponding author at: Department of Communication Engineering, Faculty of Electrical Engineering, Shahid Rajaee Teacher Training University, Tehran, Islamic Republic of Iran.
E-mail address: nbagheri@sru.ac.ir (N. Bagheri).

https://doi.org/10.1016/j.jisa.2025.104188

Available online 10 August 2025
2214-2126/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Encryption Standard). Consequently, a protocol's security weakness primarily lies in its design approach, as designers often overlook the structure of exchanged messages and their computations in lightweight protocols. However, in ultra-lightweight approaches, not only does the protocol's structure matter, but the method of encrypting information can also be a security vulnerability. In ultra-lightweight methods, a new permutation function is usually designed instead of relying on standard encryption functions, which require more hardware. The presented permutation methods typically use simple logical functions, reducing the protocol's weight. However, these same lightweight functions can potentially become vulnerabilities for the protocol. In other words, regardless of how well-designed the protocol's structure is, weaknesses in permutation functions can compromise the system's security. In this article, we thoroughly analyze a new ultra-lightweight protocol and scrutinize its security aspects to demonstrate that, while the proposed protocol's structure does not contain security flaws, its permutation function can be a potential weakness.

Bi et al. recently proposed [1] the "Secure and Efficient Two-Party Protocol Enabling Ownership Transfer of RFID Objects". This ultra-lightweight authentication protocol aims to provide a secure mechanism for transferring ownership while minimizing resource requirements and ensuring efficient operation. Ownership transfer in RFID systems refers to transferring the ownership or responsibility of an RFID-tagged item from one entity to another within the system. This process is essential in various applications where item ownership or custody must be tracked and managed efficiently.

The protocol's design is based on the principles of ultra-lightweight cryptography and efficient communication, making it suitable for resource-constrained devices and environments. It aims to ensure confidentiality, integrity, and authenticity during the ownership transfer, safeguarding sensitive information associated with the RFID objects. However, to deploy the protocol in practical scenarios, it is imperative to evaluate its security guarantees thoroughly. Hence, this paper investigates and demonstrates the security vulnerabilities of the protocol discussed in [1]. The attack methodology is applied in both computational and hardware forms within a real-world setting, employing three Raspberry Pis and a wireless network.

1.1. Motivation and contribution

Low-cost IoT devices pose significant security risks, primarily due to their lack of robust security features, making them vulnerable to cyberattacks. For instance, Microsoft researched IoT security from a customer's perspective. They highlighted that many IoT devices are not designed with security in mind, leading to significant risks when connected to the Internet. The dangers include the possibility of these devices being targeted in cyberattacks, such as the incidents involving local water treatment plants in the US, where hackers attempted to alter water treatment processes, or the Colonial Pipeline ransomware attack that caused fuel shortages across multiple states [2].

The main contributions of this paper include:

- Security analysis of the protocol proposed by Bi et al. and demonstrating its vulnerability to various attacks such as secret disclosure and desynchronization attacks.
- Prove the vulnerability and security weaknesses in the structure of permutation functions.
- Practical implementation of the attack method using three Raspberry Pi devices, with details provided in a video and shared on GitHub.

The rest of the paper presents related works in Section 2. Subsequently, the required preliminaries are provided in Section 3. Theoretical cryptanalysis of Bi et al.'s protocol is described in Section 4, while its practical implementation is detailed in Section 5. Finally, Section 6 presents the conclusion and recommendations.

2. Related works

The healthcare sector presents an interesting challenge for ultra-lightweight protocols due to the critical need for security in RFID systems. Improper use of RFID tags can threaten patient safety and privacy, making robust security measures essential. As a result, numerous research papers have been published to address these challenges.

One such protocol is CRUSAP, an ultra-lightweight RFID security authentication protocol designed for smart healthcare systems [3]. CRUSAP aims to reduce attacks by introducing a new permutation function and combining Bit-Crossing XOR rearrangement operations with a cloud server. According to the authors, security analysis and experimental results show that CRUSAP provides strong security at a lower cost. However, research has also shown that CRUSAP is vulnerable to desynchronization and secret disclosure attacks, undermining its effectiveness in certain situations [4].

Another notable protocol is ESRAS, proposed by Shariq et al. [5]. This Efficient, Secure, and Practical Ultra-lightweight RFID Authentication Scheme is tailored for low-cost tags, making it ideal for widespread healthcare applications. ESRAS improves security by using a permutation function and combining bit-wise exclusive-OR and circular left rotation. Its analysis demonstrates that ESRAS can resist known attacks, making it a suitable solution for low-cost RFID systems used in healthcare.

In the Internet of Vehicles (IoV) systems context, the AnonSURP protocol addresses RFID technology's security and privacy needs [6]. AnonSURP is an anonymous and secure RFID protocol that uses a permutation function for data encryption, ensuring user and data privacy. By protecting against common attacks, AnonSURP appears to be a promising approach to address security challenges in IoV systems.

The URASP protocol, introduced in response to the security challenges in the IoT environment, provides an ultra-lightweight RFID authentication solution [7]. URASP enhances security and privacy without adding significant computational or storage overhead, making it particularly effective for resource-constrained environments. The authors claim that URASP offers a flexible and efficient solution for improving the security of RFID systems.

The KUAUB protocol was proposed to enhance patient privacy within the Internet of Healthcare Things (IoHT) [8]. KUAUB ensures secure communication between RFID tags and readers by using a permutation function called Cro and employing pseudonyms instead of real IDs, which helps mitigate security risks. The authors claim that KUAUB provides robust protection against known attacks while safeguarding patient privacy through pseudonyms.

Fan et al. [9] introduced a cloud-based RFID authentication scheme to address security and privacy concerns in IoT systems. Their scheme uses rotation and enhanced permutation techniques for data encryption, helping to reduce the overhead on RFID tags while ensuring privacy protection.

Another recent protocol, ER2AS, uses bitwise XOR, circular left–right rotations, and an ultra-lightweight reformation operation to improve security while resisting known attacks [10]. The authors argue that ER2AS achieves heightened security with lower computation and storage overhead, making it a practical solution for real-time implementation. It effectively balances security and efficiency, which is the focus of this paper, has structural flaws in its permutation function that compromise security [1]. Despite robustness claims, these weaknesses expose the system to potential security risks.

In the mobile Internet of Medical Things (mIoMT), Deebak et al. [11] proposed the PPSA-PKV protocol. This privacy-preserving seamless authentication protocol integrates advanced cryptographic techniques to ensure secure communication between mIoMT devices in Beyond 5G-enabled healthcare systems. Their performance evaluations suggest that PPSA-PKV balances security and efficiency effectively, making it a viable solution for next-generation healthcare applications.

Table 1
Summary of related works on ultra-lightweight RFID authentication protocols.

Paper name	Methods used	Attacks	Approved by
Bi et al. [1]	Structural permutation function	Desynchronization; Secret Disclosure	This article
PPSA-PKV [11]	ECC and Hash functions	Traceability	[12]
Fan et al. [9]	Rotation, Enhanced permutation, Cloud integration	Secret Disclosure	[13]
KUAUB [8]	Cro Permutation function, Pseudonyms for privacy	Secret Disclosure	[4]
ER2AS [10]	Bitwise XOR, Circular left–right rotations, Lightweight reformation operation	No valid attacks reported yet	Non-Applicable
ESRAS [5]	Permutation function, Bitwise XOR, Circular left rotation	Secret Disclosure	[4]
URASP [7]	Permutation, Left-rotate operations	Secret Disclosure	[4]
AnonSURP [6]	Permutation function for encryption	Desynchronization	[8]
CRUSAP [3]	Permutation function, Bit-Crossing XOR, Cloud server	Desynchronization; Secret Disclosure	[4]

Table 2
Notation used in this paper.

Notation	Description
R_C	The previous (current) owner
R_N	The new (incoming) owner
T	The RFID tag
ID_o	The identifier of the previous owner
ID_n	The identifier of the incoming owner
IDS_x	The RFID tag's Pseudonym
IDS_x^{new}	The Updated pseudonym for the RFID tag
K_1, K_2, K_3	The shared keys between the tag and the previous owner
K_i^{new}	The updated shared keys after completing the protocol
K_T	Short-term shared key between tag and incoming owner
K_R	Short-term agreed key between the previous and incoming owner
r_i	The random values generated by the readers at each step
LSB	Least significant bit
MSB	Most significant bit
$PRNG$	Pseudo-random number generator
$x\|\|y$	Concatenation of x and y
Rot	Left rotation function
$f_x(x, y)$	The permutation function used in the protocol
$f_x^{-1}(x, y)$	The inverse of the f_x permutation function
$\mathcal{A}$	The adversary
$\mathbb{F}_2$	Binary field

However, Zeng et al. [12] have identified vulnerabilities in the authentication scheme, including issues with traceability and anonymity, which could expose the system to security risks.

As shown in the related works, many lightweight protocols, despite their designers' claims, have security weaknesses that can significantly impact system performance [4]. While the ER2AS protocol has not yet faced any valid attacks, issues will likely emerge over time. Given these ongoing concerns, there is a clear need for a method that matches the computational efficiency of these lightweight approaches while providing stronger security (see Table 1).

3. Preliminaries

This section introduces the preliminaries employed in this paper, including the protocol under investigation and its proposed permutation function. In the following sections, we will adhere to the notations presented in Table 2 for introducing the protocol and proposed attacks.

3.1. Permutation mechanism description

Recently, Bi et al. have proposed an ultra-lightweight protocol for secure ownership transfer of RFID objects [1]. In this efficient protocol, the tag utilizes only bitwise XOR operation, bit rotation, and a permutation function inspired by [14]. The left rotation operation is responsible for shifting string X by some bits equal to $wt(Y)(mod\ L)$, where L signifies the length of the string, and $wt(Y)$ stands for the Hamming weight of Y. This operation rotates the string to the left by a variable number of positions. The proposed permutation function, as suggested by [1], is defined as follows:

Considering two bit strings of equal length, X and Y, represented as follows:

$$X = x_1 \| x_2 \| \ldots \| x_l \tag{1}$$

$$Y = y_1 \| y_2 \| \ldots \| y_l \tag{2}$$

where $x_i, y_i \in \{0, 1\}$ for $i = 1, 2, \ldots, n$. Let the Hamming weight of Y be denoted as m. In Y, the bits that equal one are referred to as k_i. Specifically, $y_{k_1} = y_{k_2} = \cdots = y_{k_m} = 1$ and, $y_{k_{m+1}} = y_{k_{m+2}} = \cdots = yk_l = 0$.

The outcome of permuting bit string X, using this permutation function, can be represented as:

$$Z = x_{k_m} \| x_{k_{m-1}} \| \ldots \| x_{k_1} \| x_{k_l} \| x_{k_{n-1}} \| \ldots \| x_{k_{m+1}} \tag{3}$$

This operation reorders the bits in X based on the pattern of '1's in Y, producing a new bit string Z. String Y specifies the bit positions, and string X determines the permuted bit values.

3.2. The explanation of the ownership transfer protocol

The proposed protocol in [1] involves three primary entities: the former (old) owner, the incoming (new) owner, and the tag. The communication channel between the backend server and the readers is assumed to be secure and reliable. In contrast, readers and tags rely on wireless communication for data exchange, and this communication is exposed to a challenging environment highly susceptible to security breaches.

During the initial phase of the RFID system, the former and new owners, along with the tag, generate random numbers as short-term session keys within a secure and trusted environment. The former and new owners negotiate to create a pseudo-random number denoted as K_R. The new owner generates a random number K_T as a temporary shared secret and conveys it to the tag. Subsequently, the previous owner records critical information about the ownership transfer, employing the index data table.

At the commencement of the protocol, the new owner initiates by sending an initialization message to the tag, which leads to a sequence of communication steps, including checks, verifications, and key updates. The protocol claims secure and legitimate ownership transfers. During the protocol, both old and new owners engage in key exchanges, and the information they receive is verified to confirm the legitimacy of the ownership transfer. Successful verification leads to the final authorization for ownership transfer, which is executed by updating keys. Fig. 1 outlines and presents the protocol in detail.

The ownership transfer phase runs as follows:

- At the inception of the protocol, the new owner's first action is to create a random number, denoted as r_1, and calculate $A = r_1 \oplus K_T$. Subsequently, the new owner initiates the protocol by transmitting a message containing the message "*hello*" and the calculated A to the tag to initialize the protocol.
- Upon reception of this message, the tag extracts the random number r_1' from the A message and computes $B' = f_x(r_1' \oplus IDS_x, K_T)$. This calculated B' serves as the tag's response to the new owner.
- After obtaining the message B', the incoming reader, who was previously involved in generating random numbers and shared the K_T key, uses these parameters to compute $B = f_x(r_1 \oplus IDS_x, K_T)$. At this point, two distinct scenarios may unfold:

R_C $(ID_o, ID_x, K_1, K_2, K_3, K_R, IDS_x)$	R_N $(ID_o, ID_n, K_T, K_R, IDS_x)$	T $(K_1, K_2, K_3, K_T, IDS_x)$
	1. $r_1 = PRNG()$ $A = r_1 \oplus K_T$; $M_1 = \{Hello, A\}$ $\xrightarrow{2.M_1}$	
		3. $r_1' = A \oplus K_T$ $B' = f_x(r_1' \oplus IDS_x, K_T)$; $M_2 = \{IDS_x, B'\}$ $\xleftarrow{4.M_2}$
	5. $B = f_x(r_1 \oplus IDS_x, K_T)$ if $B == B'$: $r_2 = PRNG()$ $C = f_x(r_2 \oplus IDS_x, K_R)$ $D = Rot(r_2, f_x(ID_n, K_R))$; $M_3 = \{IDS_x, C, D, f_x(IDS_x, IDS_l)\}$ $\xleftarrow{6.M_3}$	
7. Continuously looping through the entire backend server if the index content matches: extract r_2' from C $D' = Rot(r_2, f_x(ID_n, K_R))$ if $D == D'$: $r_3 = PRNG()$ $E = f_x(r_3, r_2 \oplus K_R)$ $F = Rot(r_2 \oplus K_R, f_x(ID_o, r_3))$ $G = f_x(K_1 \oplus K_2, K_2 \oplus K_3)$ $H = Rot(K_2 \oplus K_3, ID_n \oplus K_R)$; $M_4 = \{E, F, G, H\}$ $\xrightarrow{8.M_4}$		
	9. extract r_3' from E $F' = Rot(r_2 \oplus K_R, f_x(ID_o, r_3'))$ if $F == F'$: extract $K_2 \oplus K_3$ from H $r_4 = PRNG()$ $I = f_x(r_4 \oplus K_T,\ K_2 \oplus K_3)$ $J = Rot(f_x(r_4, K_T), r_4 \oplus IDS_x)$; $M_5 = \{G \oplus I, J\}$ $\xrightarrow{10.M_5}$	
		11. extract r_4' from $G \oplus I$ $J' = Rot(f_x(r_4', K_T), r_4' \oplus IDS_x)$ if $J == J'$: $L = f_x(r_4 \oplus r_1, K_T) \oplus r_4$; $M_6 = \{L\}$ $\xleftarrow{12.M_6}$
	13. $L' = f_x(r_4 \oplus r_1, K_T) \oplus r_4$ if $L == L'$: $r_5 = PRNG()$ $M = f_x(r_4, K_T) \oplus r_5$ $N = Rot(r_5, f_x(r_4, K_T \oplus IDS_x))$ Update: $IDS_x^{new} = f_x(IDS_x \oplus r_4, r_5)$ $K_1^{new} = f_x(K_1 \oplus K_2, r_4) \oplus r_5$ $K_2^{new} = f_x(K_2 \oplus K_3, r_5)$ $K_3^{new} = f_x(K_T \oplus r_4, r_5)$; $M_7 = \{M, N\}$ $\xrightarrow{14.M_7}$	
		15. extract r_5' from M $N' = Rot(r_5', f_x(r_4, K_T \oplus IDS_x))$ if $N == N'$: Update: $IDS_x^{new} = f_x(IDS_x \oplus r_4, r_5)$ $K_1^{new} = f_x(K_1 \oplus K_2, r_4) \oplus r_5$ $K_2^{new} = f_x(K_2 \oplus K_3, r_5)$ $K_3^{new} = f_x(K_T \oplus r_4, r_5)$

if IDS_x^{new} is received:
$IDS_x^{old} = IDS_x^{new}$, $K_1^{old} = K_1^{new}$, $K_2^{old} = K_2^{new}$, $K_3^{old} = K_3^{new}$
if IDS_x^{old} is received:
$IDS_x^{old} = IDS_x^{old}$, $K_1^{old} = K_1^{old}$, $K_2^{old} = K_2^{old}$, $K_3^{old} = K_3^{old}$		

Fig. 1. Messages exchanged in the ownership transfer phase [1].

1. If B matches B', signifying their equality, the new owner generates a fresh random number labeled r_2. This is followed by the $C = f_x(r_2 \oplus IDS_x, K_R) calculation$. Simultaneously, D is computed as $D = Rot(r_2, f_x(ID_n, K_R))$. The combination of these calculations allows R_N to evaluate $f_x(IDS_x, ID_n)$, which, in turn, enables R_C to verify the legality of the ownership transfer. Subsequently, the R_N transmits the computed results to the former owner.
2. However, if the computed B does not align with the received B', it results in an immediate termination of the protocol.

- Upon receiving the messages from the new owner, R_C proceeds to query the index data table to find the corresponding index content based on the message $f_x(IDS_x, ID_n)$. In this context, two potential outcomes emerge:

  1. When this query's result matches, the former owner's ownership transfer request is legitimate. The next step is to verify the requester's identity. The former owner utilizes the random number r_2' extracted from the message C and the shared key K_R to recompute $D' = Rot(r_2', f_x(ID_n, K_R))$. Subsequently, a comparison is made to determine if D' equals D. If this condition holds, the former owner generates a new random number, r_3. This, in turn, leads to the calculation of multiple messages, specifically E, F, G, and H as follows:
  $E = f_x(r_3, r_2 \oplus K_R)$,
  $F = Rot(r_2 \oplus K_R, f_x(ID_o, r_3))$,

$G = f_x(K_1 \oplus K_2, K_2 \oplus K_3)$,
$H = Rot(K_2 \oplus K_3, ID_n \oplus K_R)$.
The values of E, F, G, and H are then transmitted to R_N for further processing.

2. On the contrary, if the retrieved data does not align with the expected outcome, it implies that the requesting entity does not possess the authority to change the tag ownership. Consequently, the protocol is terminated.

- Subsequently, the new owner extracts the random number r_3' from the received message E. Using this extracted value, F' is calculated as $F' = Rot(r_2 \oplus K_R, f_x(ID_o, r_3'))$. This leads to a verification process, examining the equation $F' == F$. If this verification succeeds, it allows the new owner to utilize the random number r_4 generated by the pseudo-random number generator (PRNG) and $K_2 \oplus K_3$ extracted from message E to compute the message $I = f_x(r_4 \oplus K_T,\ K_2 \oplus K_3)$. Furthermore, I is XORed with the message G to finalize the aggregation signature. To complete this step, the new owner computes J, which is calculated as $J = Rot(f_x(r_4, K_T), r_4 \oplus IDS_x)$. These two messages, $G \oplus I$ and J, are transmitted to the tag. However, the new owner must terminate the protocol if the received value does not align with the locally computed value.
- Upon reception of the verification information, the tag is responsible for extracting the random number r_4' from $G \oplus I$ by utilizing the shared key that it holds with both the old and new owners. In the subsequent phase, the tag calculates $J' = Rot(f_x(r_4', K_T), r_4' \oplus IDS_x)$. It then evaluates whether $J' == J$. When this equation is satisfied, the tag authorizes the old owner to transfer ownership. The tag computes the message $L = f_x(r_4 \oplus r_1, K_T) \oplus r_4$ as a final step. This calculated message L is then sent to the new owner.
- Upon receiving the response from the tag, the new owner embarks on the verification process to ensure the correctness of the message L. Two distinct scenarios may transpire:

  1. If the received message is authenticated, the new owner can calculate two values, namely M and N. M is computed as $M = f_x(r_4, K_T) \oplus r_5$. Simultaneously, N is obtained as $N = Rot(r_5, f_x(r_4, K_T \oplus IDS_x))$, with r_5 representing the newly generated random number. The new owner then transmits the computed messages M and N to the tag. As a concluding step, the R_N updates the tag's pseudonym and the shared keys according to the following relationships:
  $IDS_x^{new} = f_x(IDS_x \oplus r_4, r_5)$,
  $K_1^{new} = f_x(K_1 \oplus K_2, r_4) \oplus r_5$,
  $K_2^{new} = f_x(K_2 \oplus K_3, r_5)$,
  $K_3^{new} = f_x(K_T \oplus r_4, r_5)$.
  2. Conversely, if an unauthorized entity has transmitted the value of L or has been tampered with, the verification process fails.

- Following the verification stage, the tag is responsible for extracting r_5' from the received message M and computing the local value of $N' = Rot(r_5', f_x(r_4, K_T \oplus IDS_x))$. The calculated value's successful alignment with the transmitted message N indicates that the tag verified the new owner. Therefore, the tag updates the tag's pseudonym and shared keys, just like the new owner.
- If R_C receives an IDS_x^{new}, it updates its parameters; otherwise, the parameters remain the same as their previous values.

4. Security analysis of the target protocol

In this section, we delve into the vulnerabilities of the permutation function employed in Bi's protocol and introduce our attack strategies. To reduce article length and constraints in the video presentation, complete simulation details along with a full implementation video and codes have been provided on the GitHub repository: https://github.com/iman-mzm/RFID-Ownership-Transfer.

4.1. Weaknesses of the permutation function

The permutation function proposed by [1] claims that by reversing the positions of the '1' bits in the output, it overcomes the drawbacks of the function presented in [14] and provides improved security performance. Our investigations indicate that the proposed permutation function still requires improvement in its security. The weaknesses and vulnerabilities of this permutation function are as follows:

1. According to the provided explanation for the permutation function, if the first bit (MSB) of string Y, or $(y)_1$, is equal to zero, then the first bit (MSB) of string X, or $(x)_1$, after permutation, is moved to the last position (LSB). If the first bit of string Y is one, then in this case, the position of the first bit of string X can be anywhere except the last bit.
The positions of the subsequent bits can also be determined after establishing the value of the first bit of string Y. For instance, if the second bit of string Y, denoted as $(y)_2$, is equal to zero, in case the preceding bit, $(y)_1$, is also zero, bit $(x)_2$ is relocated to the second-to-last position after permutation; otherwise, $(x)_2$ is moved to the last position. However, if the value of bit $(y)_2$ is one, when $(y)_1$ is zero, bit $(x)_2$ is moved to any position except the last and second-to-last one; however, if $(y)_1$ equals one, it can be in any position except the last bit and the position of the previous bit $(x)_1$.
In other words, with the when applying $Per(x, y) = (P)_1 \| \ldots \|(P)_l$, $(x)_1$ will be assigned to $(P)_l$ or $(P)_1$ through $(P)_{n-1}$. Then, upon determining $(x)_1$, $(x)_2$ will be assigned to one of the positions $(P)_l$ and $(P)_{n-1}$, or one of the positions $(P)_1$ through $(P)_{n-2}$, and so on.
2. Similar to the first vulnerability of the permutation function, if we now look from the LSB side, if the last bit (LSB) of string Y, or $(y)_l$, is equal to one, the last bit of string X, or $(x)_l$, will be assigned to the first position after permutation. However, if the last bit (LSB) of string Y, or $(y)_l$, is equal to zero, the last bit of string X, or $(x)_l$, can be assigned to any position except the first one (MSB). The remaining bits follow a similar pattern based on the preceding bit's value.
In other words, with the presence of $Per(x, y) = (P)_1 \| \ldots \|(P)_l$, $(x)_l$ will be assigned to $(P)_1$ or $(P)_2$ through $(P)_l$. Then, upon determining $(x)_1$, $(x)_2$ will be assigned to one of the positions $(P)_1$ and $(P)_2$, or one of the positions $(P)_3$ through $(P)_l$, and so on.
3. In the context of cryptographic permutations and secure encryption, a fixed point is defined as an element within the domain of a permutation function that remains unchanged when the function is applied to it. For a given permutation function Per, an element X is considered a fixed point if $Per(X)$ equals X. Fixed points are significant in cryptographic analysis as they can introduce vulnerabilities and weaken the security of encryption schemes.
In the specific protocol under consideration, we have observed that the permutation function used exhibits a fixed point. To illustrate, let us examine the following values and patterns:

$$X = 10011, Y = 01110, Z = 10011.$$

Applying the permutation function to X leaves the output the same ($Per(10011, 01110) = 10011$). In addition to consecutive ones in string Y, other ones can be added, and likewise, besides consecutive zeros in string X, additional zeros can be included while preserving the fixed point pattern.
This observation indicates the presence of fixed points within the function. Such behavior raises security concerns, implying that specific patterns and values may remain unaffected by the permutation. This, in turn, could potentially create vulnerabilities that adversaries might exploit to perform cryptanalysis or

compromise the encryption scheme. Therefore, it is imperative to carefully consider the existence of fixed points within the permutation function during the protocol's security analysis.

Considering the vulnerabilities and weaknesses presented for the permutation function in this section and the analysis conducted by [15] for the permutation function presented in [14], it can be concluded that the mentioned permutation functions lack the necessary characteristics to preserve security. It is also evident that any permutation similar to these functions and any linear combination of them will exhibit vulnerabilities identical to the ones discussed and will not be secure.

We will leverage the first vulnerability introduced in the following subsections and present our attacks.

4.2. Desynchronization attack

A desynchronization attack is a calculated and malicious effort to disturb critical protocol components' precise timing and alignment. These components include the timely updates of cryptographic keys and other crucial elements necessary for the protocol's proper operation. A desynchronization attack aims to introduce misalignment or inconsistencies among these elements, which could undermine the protocol's security and overall effectiveness. Therefore, addressing desynchronization attacks is paramount in safeguarding the integrity and dependability of the specific protocol under scrutiny.

The protocol under investigation has a contingency plan for potential disruptions caused by a desynchronization attack. This entails storing critical data from two previous communication rounds, aiming to ensure a robust defense against adversarial interference by keeping tag's IDS_{old} and K_{old} in the current reader's memory during subsequent interactions.

In Step 13 of Fig. 1, upon receiving the message L from the tag, the new owner compares its value with the locally calculated one. In the event of equality, the new owner calculates the values of M and N. This calculation involves the utilization of a random value, r_5, and follows the relationships defined by equations $M = f_x(r_4, K_T) \oplus r_5$ and $N = Rot(r_5, f_x(r_4, K_T \oplus IDS_x))$. Subsequently, the new owner performs its final actions in the session, which include updating the values of K_1, K_2, K_3, and the tag's ID (IDS_x). After these updates, the new owner transmits the messages M and N to the tag.

During this time, an attacker can block the messages sent by the new owner, preventing the tag from receiving the values of M and N. With this action, the tag, without receiving the messages M and N, does not update its parameters, even though the new owner has updated its own. Additionally, the old owner does not update its values if it does not receive IDS_x^{new}, leaving its parameters unchanged.

As a result, by eavesdropping on all messages and blocking messages M and N, the attacker maintains the tags and the old owner's parameters as their old values, even though the new owner has updated its parameters. After updating its values, R_N assumes it is the new owner, but this does not occur without updating the tag's parameters and R_C. After this point, the protocol lacks any authentication stages. The attacker can be anyone, including the old owner. In Table 3, we can observe the values of K_1, K_2, K_3, and IDS_x for all three identities after the desynchronization attack.

Another approach to desynchronize the tag from the reader is to perform a replay attack following the method described in [16], given that the tag does not contribute to the protocol's freshness. Although this approach has higher complexity, it also demonstrates that the target protocol suffers from replay attacks.

4.3. Full secret disclosure attack

This section presents a full secret disclosure attack, allowing an attacker to recover a hidden value of length l by conducting $l + 1$ sessions with the tag. The attacker can be anyone, including the old owner R_C. As a result of the attack, the attacker can completely recover the secret value "K_T", which is the session key shared between the new owner and the tag, by executing $l + 1$ sessions.

Table 3
Internal secret values of each entity after desynchronization.

R_C	R_N	T
IDS_x^{old}	$IDS_x^{new} = f_x(IDS_x \oplus r_4, r_5)$	IDS_x^{old}
K_1^{old}	$K_1^{new} = f_x(K_1 \oplus K_2, r_4) \oplus r_5$	K_1^{old}
K_2^{old}	$K_2^{new} = f_x(K_2 \oplus K_3, r_5)$	K_2^{old}
K_3^{old}	$K_3^{new} = f_x(K_T \oplus r_4, r_5)$	K_3^{old}

Considering Fig. 1, the protocol begins by sending the "*hello*" message and the value A, calculated by R_N, to the tag. The tag first recovers the random value r'_1 and, after calculating the value $B' = f_x(r'_1 \oplus IDS_x, K_T)$, sends it along with its ID to R_N upon receiving the message sent by R_N. By exploiting the protocol's initial three steps, the attacker can execute the attack and disclose the value of K_T. The proposed attack will proceed as follows:

Since the tag is passive and the reader solely controls the initiation and termination of a session, the attacker can start a new session with the same secret session parameters. The attacker generates a random value, denoted as "A", with a length of "l" and includes "A" in the "*hello*" message sent to the tag. Upon receiving this message, the tag extracts r'_1 and calculates $B' = f_x(r'_1 \oplus IDS_x, K_T)$. The tag then sends B' and its identifier back to the attacker.

Considering e_i as standard basis vectors in $\mathbb{F}_l^2$, in this stage, the attacker adds the value A to e_1 to generate A_1, meaning $A_1 = A \oplus e_1$. Subsequently, the attacker sends A_1 and a "*hello*" message to the tag. Upon receiving the message from the attacker, the tag, after extracting $r'_{1,1}$ and computing $B'_1 = f_x(r'_{1,1} \oplus IDS_x, K_T)$, sends B'_1 along with its identifier to the attacker.

Here, since the value of A_1 differs from A in only one bit (the first bit), considering the computations performed by the tag, the value of B'_1 will also vary from B' in only one bit. By exploiting the first weakness of the permutation function mentioned in subsection A and considering that the calculation of different B'_i values is done by the tag using the relationship $B'_i = f_x(r'_{1,i} \oplus IDS_x, K_T)$, the attacker, in this stage, compares the values of B' and B_1 with each other. If these two values differ in the last bit (LSB), the attacker concludes that the first bit (MSB) of the K_T string is equal to zero, and otherwise (i.e., when there is a difference in any other bit except the last one), the first bit (MSB) of the K_T string is equal to one. As a result, the first bit of K_T will be recovered.

With the knowledge of the first bit (MSB) and to obtain the second bit of the K_T string, the attacker, this time, adds the value of A to e_2, obtaining $A_2 = A \oplus e_2$. By sending A_2 and "*hello*" to the tag and receiving the message B'_2 from the tag, the attacker, once again, compares the value of B' with B'_2. Because A and A_2 only differed in the second bit, B' and B'_2 will also differ in only one bit. Now, considering the first weakness mentioned for the permutation function, if the first bit of K_T is zero and there is a difference between B' and B'_2 in the second bit counting from the end, the second bit of K_T is also zero, and otherwise, it is one. However, if the first bit of K_T is one and there is a difference between B' and B'_2 in the last bit (LSB), the second bit of K_T is zero, and otherwise, it is one.

After knowing the first and second bits of K_T, the attacker can similarly recover the third bit of K_T. In general, in each session, the attacker adds the values of A and e_i together, calculates the value A_i, sends it to the tag, and upon receiving B'_i, compares it with B'. By repeating this process for each bit position and knowing the previous bits (i.e., bits 1 to $i - 1$), the attacker determines the ith bit of K_T, which has a length of l. As a result, executing $l + 1$ sessions makes all l bits of the secret value K_T known to the attacker. Based on simulations conducted in a Python environment, with l set to 128, this process only takes 30 milliseconds.

The pseudocode of the attack is presented in Algorithm 1. According to this algorithm, after the initial setup phase and sharing of secret values among the entities in the protocol, the attacker initiates a session by sending a random value, A, and a "*hello*" message to the tag. After the tag calculates and sends the value B' to the attacker, the attacker initializes an initial value of all ones for K_T. During a loop with l sessions, in each session, the attacker adds the value A to the standard basis vector e_i and sends it to the tag, receiving the value B'_i from the tag. The attacker then calculates the difference between B' and B'_i. Afterward, the attacker counts the number of zeros in the K_T string from the first bit (MSB) to the $(i-1)$-th bit and names this count as m. If the $(l-1)-m$-th bit (from the left) of the difference between B' and B'_i equals one, the attacker concludes that the ith bit of the secret K_T value is zero; otherwise, it is one.

Algorithm 1: Disclosure Attack Algorithm

Data: Initialization phase data
Result: K_T
Attacker: $A \leftarrow PRNG(l)$;
Tag: $r'_1 \leftarrow A_1 \oplus K_T$;
$B' \leftarrow f_x(r'_{1,1} \oplus IDS_x, K_T)$;
Attacker: $K_T \leftarrow [1] \times l$;
for *i in range(l)* **do**
  $e_i \leftarrow$ Standard basis vector;
  $A_i \leftarrow A \oplus e_i$;
  /*Send $(A_i, Hello)$ to the tag */
  /*Attacker receives B'_i */
  $delta \leftarrow B' \oplus B'_i$;
  $m \leftarrow KT_R[:i].count(0)$;
  if $delta[(l-1)-m] == 1$ **then**
    $K_T[i] \leftarrow 0$;
  else
    $K_T[i] \leftarrow 1$;
  end
end

4.4. Violation of the new owner's privacy

Ensuring the privacy of both the previous and new owners presents a novel challenge in ownership transfer protocols. It is imperative that neither the adversary nor the previous owner can make conjectures about the private information of the new owner. In [1], it has been claimed that in the designed protocol, the old owner cannot guess the new shared keys of the incoming owner and track the tag. However, contrary to this claim, here we present and simulate an attack in which the old owner can compute the updated secret parameters of the new owner and the tag. As a result, the new owner's privacy will be compromised entirely.

In this attack, if the previous owner R_C is considered the attacker, after recovering the session key between the tag and the new owner (K_T) in the disclosure attack, the attacker can eavesdrop on the messages $G \oplus I$ and M that the new owner sends to the tag. The attacker can extract the previously generated random values r_4 and r_5 from the new owner. With knowledge of r_4 and r_5, and having K_T, the attacker, or the previous owner, can calculate the tag's ID and the updated keys of the new owner. The attack details will be explained further.

The previous owner participates in the main ownership transfer session and follows the protocol steps to track the tag and violate the new owner's privacy. Based on the disclosure attack described in the previous section, the attacker, or the previous owner, deduces and obtains the secret value K_T by conducting separate sessions with the tag.

Also, during this process, due to the lack of security in the communication channel between the tag and the new owner, the old owner eavesdrops on two messages, $G \oplus I, J$, and M, N, sent by the new owner to the tag in Steps 10 and 14 of the main session. The attacker, in this case, only needs the messages $G \oplus I$ and M, which are calculated based on the $G \oplus I = f_x(K_1 \oplus K_2, K_2 \oplus K_3) \oplus f_x(r_4 \oplus K_T, K_2 \oplus K_3)$ and $M = f_x(r_4, K_T) \oplus r_5$ relationships.

At this stage, the attacker only needs to extract the random values r_4 and r_5 generated by the new owner from the two messages mentioned above. The extraction of the value r_4 from the $G \oplus I$ message by the previous owner can be accomplished as follows, given the values K_1, K_2, K_3, and K_T and utilizing the homomorphic property of the permutation function f_x, based on Eqs. (4) and (5).

$$C = f_x^{-1}(G \oplus I, K_2 \oplus K_3) \tag{4}$$

$$r_4 = C \oplus K_1 \oplus K_2 \oplus K_T \tag{5}$$

The random value r_5 can also be easily recovered by the previous owner, given K_T and r_4, using Eq. (6).

$$r_5 = M \oplus f_x(r_4, K_T) \tag{6}$$

After obtaining the key K_T and the two random values r_4 and r_5 and having access to the old tag ID and keys, the previous owner can quickly compute the new tag ID and the updated keys of the new owner based on Eqs. (7) to (10).

$$IDS_x^{new} = f_x(IDS_x \oplus r_4, r_5) \tag{7}$$

$$K_1^{new} = f_x(K_1 \oplus K_2, r_4) \oplus r_5 \tag{8}$$

$$K_2^{new} = f_x(K_2 \oplus K_3, r_5) \tag{9}$$

$$K_3^{new} = f_x(K_T \oplus r_4, r_5) \tag{10}$$

4.5. Critique of the previous BAN logic security proof

The security analysis of the protocol proposed in [1] includes a formal proof using the Burrows–Abadi–Needham (BAN) logic. While BAN logic is a valuable tool for analyzing the beliefs of protocol participants regarding communication properties, its application has specific limitations. BAN logic primarily focuses on the flow of information and the resulting beliefs of trusted parties within the protocol execution. It is not designed to directly model or detect vulnerabilities stemming from adversarial actions such as key compromise or replay attacks that exploit protocol state inconsistencies or secret disclosure. Furthermore, BAN logic typically assumes that protocol participants are trustworthy and does not inherently reason about scenarios where secrets are exposed to an external adversary or where parties are desynchronized.

Our analysis reveals that the BAN logic proof presented in [1, Sec. VI] relies on several assumptions and derivations that are invalidated by the attacks we have identified, particularly the desynchronization and secret disclosure attacks. Consequently, the conclusions drawn from their BAN analysis regarding the protocol's security are not valid.

Specifically, we highlight the following critical issues in their BAN proof:

1. **Invalid Freshness Assumptions:** The proof makes assumptions about the freshness of received messages that are not supported by the protocol design or the properties of BAN logic. For instance, assumption A4 states that the Tag T believes that r_4 is fresh ($T \mid\equiv \sharp(r_4)$). However, r_4 is generated by R_N, not T. In BAN logic, a party can typically only assume the freshness of a nonce it generated itself or has recently seen in a mechanism guaranteeing freshness. Given that T does not contribute to the generation of r_4 (or r_1, r_2, r_3, r_5), its belief in their freshness is an unsupported assumption. This flawed assumption is used in derivation steps such as V11, V13, V14, and V15, which subsequently lead to incorrect conclusions about T's beliefs regarding message (e.g., V12: $T \mid\equiv R_N \mid\equiv J$). Our described replay attacks directly exploit scenarios where replayed messages containing old nonces are accepted due to the lack of proper freshness verification by T, demonstrating that T cannot reliably believe in the freshness of received messages originating from R_N or other entities when these nonces were not generated by T itself.

2. **Invalid Symmetric Key Assumptions:** Assumptions A5 through A11 assert the existence of shared symmetric keys between legitimate parties (e.g., $R_N \overset{K_T}{\longleftrightarrow} T$). These assumptions are fundamental to many derivation steps in their proof, enabling parties to believe in the content and origin of encrypted messages. However, our identified desynchronization and secret disclosure attacks demonstrate that an adversary can compromise or reveal shared secret keys K_T. Once a key is compromised, the assumption that only legitimate entities share it becomes false. A BAN proof built upon the premise of uncompromised, exclusively shared symmetric keys cannot hold in the presence of attacks that invalidate this premise. The reliance on assumptions A5-A11, which presuppose keys known only to legitimate entities, renders the derivations based on them (including those contributing to goals G1-G5) invalid under the conditions of our described attacks. BAN logic itself is not equipped to detect such external key compromise, and attempting to prove security goals like G2 (related to secret information release) using BAN is outside its intended scope.

In summary, the BAN logic proof in [1] is fundamentally flawed because it relies on assumptions regarding message freshness and the secrecy of symmetric keys that are directly contradicted by the vulnerabilities we expose through desynchronization and secret disclosure attacks. Therefore, the proof does not provide a valid guarantee of the protocol's security properties. For clarity, we refer the interested reader to [1, Sec. VI, p. 16233] for the original presentation of their BAN proof.

4.6. Mathematical formal security verification

Following the provided weaknesses in Section 4.2, Section 4.3, and Section 4.4, in this section we provide a mathematical formal security analysis of the Bi et al. RFID ownership transfer protocol. The provided analysis is based on the following protocol's weakness hierarchy:

$$\begin{cases} \text{Permutation Linearity} \Rightarrow \text{Secret Disclosure} \\ \text{Lack of State Continuity} \Rightarrow \text{Desynchronization} \\ \text{Information Leakage} \Rightarrow \text{Privacy Violation} \end{cases}$$

Through our analysis we evaluate the protocol under a Dolev-Yao [17] adversary $\mathcal{A}$ which is capable of:

- Eavesdropping, modifying, blocking, or replaying messages
- Initiating unlimited protocol sessions
- Exploiting algebraic properties of cryptographic primitives

Let $|x|$ denote the bit length of string x. For $Per(x, y)$, let $|x| = |y| = |Z| = l$, where $Z = Per(x, y) = f_x(x, y)$ is the output string. We denote the position of the ith bit of x in Z for a given y as $\text{pos}((x)_i, Z) \in \{1, \dots, l\}$.

The analysis in Section 4.1 reveals specific predictable behaviors and a lack of cryptographic randomness in the $f_x(x, y)$ function. We formalize the relevant vulnerability of the protocol.

Lemma 1 (*Predictable Bit Movement Property of $Per(\cdot)$ for $pos((x)_i, Z)$*)**.** *Assuming $y \neq 1^l$, the position of the ith bit of x, $(x)_i$, in the output Z, denoted $pos((x)_i, Z)$, is deterministically or highly predictably determined by the values of the bits of y from 1 to i, i.e., $(y)_1, \dots, (y)_i$.*

Proof. This lemma formalizes the Per function's behavior based on the analysis in Section 4.1. The proof relies on reverse-engineering the Per function's construction, which moves $(x)_1$ to position l when $(y)_1 = 0$, and applies similar rules for other bits.

Given inputs x and y of length l, the permutation $f_x(x, y) = Z$ exhibits predictable behavior:

1. The position of $(x)_1$ in Z is determined by $(y)_1$:
   - If $(y)_1 = 0$, then $\text{pos}((x)_1, Z) = l$ (LSB position)
   - If $(y)_1 = 1$, then $\text{pos}((x)_1, Z) \in \{1, \dots, l-1\}$
2. Given $\text{pos}((x)_j, Z)$ and $(y)_j$ for $1 \leq j \leq i-1$, $\text{pos}((x)_i, Z)$ is determined by $(y)_i$:
   - If $(y)_i = 0$, $\text{pos}((x)_i, Z)$ is determined deterministically
   - If $(y)_i = 1$, $\text{pos}((x)_i, Z)$ is assigned non-deterministically to an unoccupied position
3. The position of $(x)_l$ in Z is determined by $(y)_l$:
   - If $(y)_l = 1$, then $\text{pos}((x)_l, Z) = 1$ (MSB position)
   - If $(y)_l = 0$, then $\text{pos}((x)_l, Z) \in \{2, \dots, l\}$

This predictable dependence demonstrates a lack of diffusion and confusion properties required for secure permutations, completing the proof. □

Lemma 2 (*Determining $pos((x)_i, Z)$*)**.** *Assume $x \oplus x' = e_i$. Given $f_x(x, y)$ and $f_x(x', y)$, the position of the ith bit of x, $(x)_i$, in the output Z, denoted $pos((x)_i, Z)$, is determined deterministically.*

Proof. Following the definition of $f_x(x, y)$, we have $f_x(x, y) \oplus f_x(x', y) = f_x(x \oplus x', y)$. Given $x \oplus x' = e_i$:

$$f_x(x, y) \oplus f_x(x', y) = f_x(e_i, y)$$

which uniquely determines $\text{pos}((x)_i, Z)$ since $f_x(e_i, y)$ has a single 1 at position $\text{pos}((x)_i, Z)$ and 0 elsewhere. □

Theorem 1 (*Secret Disclosure*)**.** *In the Bi et al. RFID ownership transfer protocol, the permutation function f_x leaks K_T in $l+1$ sessions.*

Proof.

(1) Let $K_T = (b_1, \dots, b_l) \in \mathbb{F}_2^l$. Adversary $\mathcal{A}$:

- Sends $A \in \mathbb{F}_2^l$ and observes $M_2 = \{IDS_x, B\}$ returned by T, where $B = f_x(r_1 \oplus IDS_x, K_T)$ and $r_1 = A \oplus K_T$
- For $1 \leq i \leq l$:
  (a) Sends $A_i = A \oplus e_i$ and observes $M_2^i = \{IDS_x, B_i\}$ returned by T, where $B_i = f_x(r_1^i \oplus IDS_x, K_T)$ and $r_1^i = A_i \oplus K_T$
  (b) Given $r_1 = A \oplus K_T$, $r_1^i = A_i \oplus K_T$, and $A_i = A \oplus e_i$:

  $$r_1^i \oplus r_1 = e_i$$

  $$B \oplus B_i = f_x(e_i, K_T)$$

  This determines $\text{pos}((e_i)_i, f_x(e_i, K_T))$ by Lemma 2, which equals $\text{pos}((r_1 \oplus IDS_x)_i, Z)$ in the original permutation. Using Lemma 1, the adversary determines $(K_T)_i$ from this position and previously learned bits $\{(K_T)_j\}_{j=1}^{i-1}$.

The attack succeeds with probability 1 except when $K_T \in \{0^l, 1^l\}$, where 0^l and 1^l are indistinguishable due to degenerate permutation behavior. □

Theorem 2 (*Desynchronization Attack*)**.** *In the Bi et al. RFID ownership transfer protocol, an adversary $\mathcal{A}$ can desynchronize the tag T and the new owner R_N with probability 1 by blocking the final update.*

Proof.

1. Let the initial protocol states be:

$$\sigma_T = (K_1, K_2, K_3, K_T, IDS_x), \quad \sigma_{R_N} = (IDS_x, K_1, K_2, K_3, K_T, K_R)$$

$$\sigma_{R_C} = (IDS_x^{old}, K_1^{old}, K_2^{old}, K_3^{old}, IDS_x, K_1, K_2, K_3, K_R)$$

2. After Step 14, $\mathcal{A}$ blocks messages M and N. The updated states become:

$$\sigma_T = (IDS_x, K_1, K_2, K_3, K_T)$$

$$\sigma_{R_N} = (IDS_x^{new}, K_1^{new}, K_2^{new}, K_3^{new}, K_T, K_R)$$

where

$$\begin{aligned} IDS_x^{new} &= f_x(IDS_x \oplus r_4, r_5) \\ K_1^{new} &= f_x(K_1 \oplus K_2, r_4) \oplus r_5 \\ K_2^{new} &= f_x(K_2 \oplus K_3, r_5) \\ K_3^{new} &= f_x(K_T \oplus r_4, r_5) \end{aligned}$$

3. Subsequent authentication fails because:

$$\Pr\left[f_x(IDS_x, K_T) = f_x(IDS_x^{new}, K_T^{new})\right] = 2^{-l}$$

The tag will fail authentication with the new owner with probability $1 - 2^{-l}$ following this attack. □

Theorem 3 (*Violation of new Owner Privacy*). *In the Bi et al. RFID ownership transfer protocol, the old owner R_C can compute the new keys $K_1^{new}, K_2^{new}, K_3^{new}$.*

Proof. From the public channel the R_C can capture $G \oplus I$ and M, where $G \oplus I = f_x(K_1 \oplus K_2, K_2 \oplus K_3) \oplus f_x(r_4 \oplus K_T, K_2 \oplus K_3) = f_x(K_1 \oplus K_2 \oplus r_4 \oplus K_T, K_2 \oplus K_3)$ and $M = f_x(r_4, K_T) \oplus r_5$, then

1. From intercepted messages $G \oplus I$ and M, R_C computes:

$$\begin{aligned} r_4 &= f_x^{-1}(G \oplus I, K_2 \oplus K_3) \oplus K_1 \oplus K_2 \oplus K_T \\ r_5 &= M \oplus f_x(r_4, K_T) \end{aligned}$$

2. R_C then computes the updated parameters:

$$\begin{aligned} IDS_x^{new} &= f_x(IDS_x \oplus r_4, r_5) \\ K_1^{new} &= f_x(K_1 \oplus K_2, r_4) \oplus r_5 \\ K_2^{new} &= f_x(K_2 \oplus K_3, r_5) \\ K_3^{new} &= f_x(K_T \oplus r_4, r_5) \end{aligned}$$

Since R_C knows K_1, K_2, K_3, K_T and has extracted r_4 and r_5:

$$\Pr\left[\text{Successful recovery of } K_i^{new}\right] = 1 \quad \square$$

5. Implementation in the experimental environment

This section presents the protocol's implementation and vulnerabilities in a realistic setting. The experimental environment consists of three Raspberry Pi 3 Model v1.2 devices, which act as the tag, the new owner, and the old owner nodes. As shown in Fig. 2 and ilustated in Fig. 3, the Raspberry Pis are connected to a common access point via Wi-Fi network cards. All nodes are on the same subnet with a netmask of 255.255.255.0, enabling them to transmit packets among each other. The protocol operations, such as packet transmission, are performed over Wi-Fi. In addition, an HP Pavilion DM3 Notebook laptop is connected to these nodes via an Ethernet network card and a switch. The computer functions as the command and control center. The OpenSSH [18] program on the Raspberry Pi OS (64-bit) operating system Pis is connected via Ethernet cables and a switch. The Raspberry Pi runs the recommended operating system, Raspberry Pi OS (64-bit). The laptop runs the Ubuntu 20.04 LTS operating system. The implementation is done in Python, using the socket library to transfer the protocol packets. The packets are transmitted using the TCP protocol at the transport layer. This setup provides a controlled yet realistic environment for testing the protocol's functionality, performance, and security. The combination of wireless and wired communication, secure command execution via SSH, and a standardized operating system environment ensures comprehensive protocol evaluation under various conditions. .

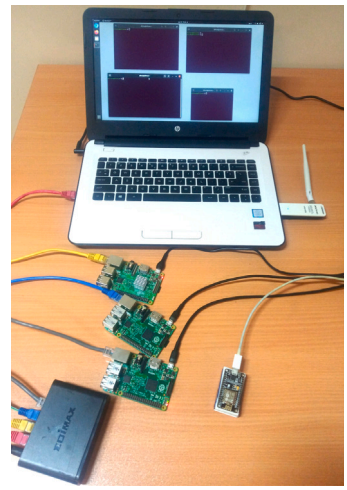

Fig. 2. Hardware components of implementation.

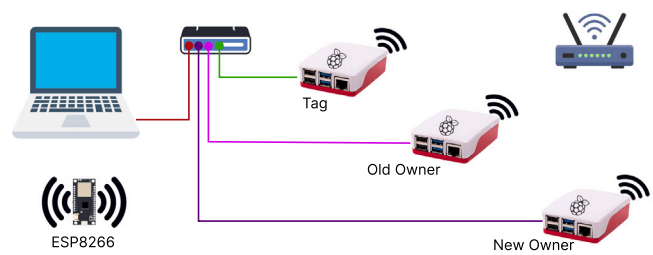

Fig. 3. Network topology.

5.1. Desynchronization attack

As mentioned, this attack aims to block the message that carries the payloads M and N. This message is sent from the new owner to the tag in Step 14 of the protocol. To block this message, the attacker utilizes a Denial-of-Service (DoS) technique known as the de-authentication attack. This method forces a targeted device to disconnect from its access point by exploiting vulnerabilities in the Wi-Fi protocol. The attack is carried out using the ESP8266 Deauther [19] library, a tool specifically designed to send de-authentication packets over a Wi-Fi network. By continuously transmitting these packets, the attacker disrupts the communication channel between the devices, preventing the message from being delivered. In our setup, the Deauther library is installed on the ESP8266 board and runs by default at 192.168.4.1. The attacker can launch the de-authentication attack by sending the appropriate requests to 192.168.4.1. In this scenario, the attacker, assumed to be the laptop, gains the ability to monitor all the packets transmitted across the network by configuring the Wireless Network Interface Controller (NIC) into monitor mode. This mode allows the NIC to capture all packets within its range, regardless of whether they are addressed to the attacker's device. The attacker bypasses normal Wi-Fi communication restrictions by enabling monitor mode and obtains a comprehensive view of the network's data traffic. This includes observing packet headers, payloads, and unsecured communications, which can be analyzed further for sensitive information or potential exploits. Moreover, the attacker knows that the message that carries the payloads M and N is sent in Step 14 of the protocol. With these capabilities, the attacker can prevent the delivery of M and N.

5.2. Full secret disclosure attack

In this attack, the attacker can be any party, including the old owner. In this scenario, we assume the attacker is the old owner, who registered as a legitimate node in the network. By leveraging their prior knowledge of the system and exploiting protocol vulnerabilities, the old owner can retrieve the shared secret key between the tag and the new owner. This is achieved by initiating $l+1$ communication sessions with the tag.

5.3. Violation of the new owner's privacy attack

In this scenario, the attacker utilizes a laptop as a tool to execute the attack. The laptop, equipped with a Wireless Network Interface Controller (NIC), is configured in monitor mode, enabling it to intercept and capture all packets transmitted within the network, regardless of their intended destination. The attacker is assumed to be the old owner, who had prior access to the system and now exploits this position to violate the new owner's privacy. The success of this attack hinges on completing all the steps of the protocol. By carefully monitoring the network traffic, the attacker identifies and captures two crucial values: $G \oplus I$ and M, which are embedded in the payloads of the messages transmitted during Steps 10 and 14, respectively. Once the laptop has intercepted these values, it transmits them over a Wi-Fi channel to the old owner. Meanwhile, leveraging their knowledge of the system and using the full secret disclosure attack described earlier, the old owner extracts the shared key between the tag and the new owner. With access to $G \oplus I$, M, and the shared key, the old owner now possesses all the information needed to compute the concealed and sensitive values exchanged between the tag and the new owner.

6. Future directions and conclusion

This paper examined an ultra-lightweight authentication protocol based on permutation functions. Initially, we demonstrated through computational methods that the proposed protocol is susceptible to various attacks, including those that result in secret disclosure and desynchronization. We then investigated the underlying causes of these vulnerabilities and conducted a comprehensive security analysis of the employed permutation function. The findings indicated that while the overall structure of the proposed protocol exhibits reasonable resilience, the intrinsic deficiencies in the permutation function render the entire system susceptible to vulnerability. We conducted a simulation using three Raspberry Pi devices to validate the proposed attack method's practicality further and provided detailed results. The protocol proposed by Bi et al. is one example of numerous ultra-lightweight protocols. Generally, an authentication protocol must consider various parameters to achieve security and lightweight properties. Given that the permutation function primarily focuses on reducing the protocol's weight and the functions it uses, it fails to meet all the requirements of a secure structure.

The simple utilization of shift functions and logical operators, such as XOR, enables attackers to compromise the protocol's security through repetitive computations and relatively low-powered tools, such as Raspberry Pi devices. Conversely, a lightweight design is crucial for IoT devices in the present era, necessitating a delicate equilibrium between security and weight.

Utilizing lightweight cryptographic methods is the recommended solution to address the issues with permutation-based protocols while maintaining hardware and energy constraints. Specifically, in protocols where the employed security primitive structure is inadequate, lightweight cryptographic functions, such as ASCON [20], can serve as effective solutions. However, using a secure encryption function alone does not address all the security concerns inherent in the protocol examined in this study.

More precisely, the protocol under investigation has a structural flaw, which stems from the fact that the tag does not contribute to the protocol's randomness. As a result, an adversary could replay eavesdropped messages in subsequent sessions. In this regard, a recent study [16] examined several lightweight and ultra-lightweight protocols designed based on similar strategies and found that they all displayed similar vulnerabilities. As part of the solutions proposed in the study, the authors introduced an alternative structure for such protocols and employed robust, lightweight cryptographic functions.

Thus, for any possible improvements to the protocol by Bi et al. the structure of the messages should be modified to ensure that all parties involved in the protocol contribute to the freshness of each session. In doing so, assuming all other security requirements are met, adversaries cannot reuse captured messages in later sessions. However, achieving this also necessitates providing a reliable mechanism for generating random values within each entity, which increases the tag's cost. Ultimately, it must be acknowledged that security cannot be achieved without incurring additional costs.

CRediT authorship contribution statement

Iman Mirzaali: Writing – review & editing, Writing – original draft, Software, Investigation, Data curation, Conceptualization. **Alireza Javadi:** Writing – review & editing, Writing – original draft, Visualization, Software, Investigation, Conceptualization. **Sadegh Sadeghi:** Writing – review & editing, Project administration, Methodology, Investigation, Formal analysis, Conceptualization. **Payman Pahlevani:** Writing – review & editing, Resources, Investigation, Formal analysis, Conceptualization. **Nasour Bagheri:** Writing – review & editing, Supervision, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization. **Ygal Bendavid:** Writing – review & editing, Validation, Supervision, Resources, Investigation, Conceptualization. **Samad Rostampour:** Writing – review & editing, Validation, Supervision, Software, Project administration, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare no conflict of interest. The founding sponsors had no role in the design of the study; in the collection, analysis, or interpretation of data; in the writing of the manuscript, and in the decision to publish the results.

Acknowledgments

We thank Dr. Masoumeh Safkhani for her critical feedback and analytical work on the formal security verification components of this paper, specifically addressing the Reviewer's recommendations regarding formal analysis methods (Sections 4.5 and 4.6). Her insights enhanced the completeness of our security evaluation. Nasour Bagheri acknowledges the financial support received from Shahid Rajaee Teacher Training University, Iran under grant number 1404/385248.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.jisa.2025.104188.

Data availability

Data will be made available on request.

References

[1] Bi Y, Fan K, Zhang K, Bai Y, Li H, Yang Y. A secure and efficient two-party protocol enabling ownership transfer of RFID objects. IEEE Internet Things J 2023;10(18):16225–37.

[2] Institute P. The state of IoT/OT cybersecurity in the enterprise. 2021, https://aka.ms/mdiotponemon2021. Last (Accessed 2024).

[3] Wang X, Fan K, Yang K, Cheng X, Dong Q, Li H, et al. A new RFID ultra-lightweight authentication protocol for medical privacy protection in smart living. Comput Commun 2022;186:121–32.

[4] Hosseinzadeh M, Servati MR, Rahmani AM, Safkhani M, Lansky J, Janoscova R, et al. An enhanced authentication protocol suitable for constrained RFID systems. IEEE Access 2024.

[5] Shariq M, Singh K, Lal C, Conti M, Khan T. ESRAS: an efficient and secure ultra-lightweight RFID authentication scheme for low-cost tags. Comput Netw 2022;217:109360.

[6] Shariq M, Singh K, Maurya PK, Ahmadian A, Taniar D. AnonSURP: an anonymous and secure ultralightweight RFID protocol for deployment in internet of vehicles systems. J Supercomput 2022;1–26.

[7] Shariq M, Singh K, Maurya PK, Ahmadian A, Ariffin MRK. URASP: an ultra-lightweight RFID authentication scheme using permutation operation. Peer- To-Peer Netw Appl 2021;14:3737–57.

[8] Khan MA, Ullah S, Ahmad T, Jawad K, Buriro A. Enhancing security and privacy in healthcare systems using a lightweight RFID protocol. Sensors 2023;23(12):5518.

[9] Fan K, Luo Q, Zhang K, Yang Y. Cloud-based lightweight secure RFID mutual authentication protocol in IoT. Inform Sci 2020;527:329–40.

[10] Kumar A, Singh K, Shariq M, Lal C, Conti M, Amin R, et al. An efficient and reliable ultralightweight RFID authentication scheme for healthcare systems. Comput Commun 2023;205:147–57.

[11] Deebak B, Hwang SO. Privacy preserving based on seamless authentication with provable key verification using miomt for b5g-enabled healthcare systems. IEEE Trans Serv Comput 2024.

[12] Zeng D, Badshah A, Tu S, Waqas M, Han Z. A security-enhanced ultra-lightweight and anonymous user authentication protocol for telehealthcare information systems. IEEE Trans Mob Comput 2025.

[13] Nagarajan M, Rajappa M. Simple yet secure encoder architecture and ultra-lightweight mutual authentication protocol for RFID tags in IoT. J Circuits Syst Comput 2023;32(07):2350118.

[14] Tian Y, Chen G, Li J. A new ultralightweight RFID authentication protocol with permutation. IEEE Commun Lett 2012;16(5):702–5.

[15] Bagheri N, Safkhani M, Peris-Lopez P, Tapiador JE. Weaknesses in a new ultralightweight RFID authentication protocol with permutation—RAPP. Secur Commun Netw. 2014;7(6):945–9.

[16] Safkhani M, Rostampour S, Bendavid Y, Sadeghi S, Bagheri N. Improving RFID/IoT-based generalized ultra-lightweight mutual authentication protocols. J Inf Secur Appl 2022;67:103194.

[17] Dolev D, Yao A. On the security of public key protocols. IEEE Trans Inf Theory 1983;29(2):198–208.

[18] OpenSSH. Openssh openssh. 2024, https://www.openssh.com. (Accessed 18 February 2024).

[19] Technologies S. Esp8266 deauther. 2024, https://github.com/spacehuhntech/esp8266_deauther. (Accessed 18 February 2024).

[20] Dobraunig C, Eichlseder M, Mendel F, Schläffer M. Ascon v1. 2: lightweight authenticated encryption and hashing. J Cryptology 2021;34:1–42.